PROJECT DOCUMENTATION REPORT

ORGANIZATIONAL KNOWLEDGE GAP INTELLIGENCE PLATFORM

A Full-Stack Enterprise Knowledge Intelligence and Learning Platform

FINAL PROJECT DOCUMENTATION

Submitted by

Harshitha Shri L

lhar.it2024@rmd.ac.in

Team 1

Batch 1

Abstract & Executive Summary

The Organizational Knowledge Gap Intelligence Platform is a full-stack enterprise application designed to continuously identify, quantify and reduce knowledge and skill gaps across employees, roles and departments. The central problem addressed by the platform is the disconnect between the skills an organization expects and the skills employees currently possess. Traditional training systems commonly operate as catalogs of courses, while capability intelligence requires a continuous evidence-based cycle that connects competency benchmarks, assessments, learning activity and business needs.

The platform combines a React.js frontend with a Spring Boot backend and PostgreSQL as the primary relational store. The demonstrated system provides employee profiles, skill inventories, competency benchmarking, knowledge-gap analysis, heatmap visualization, training recommendations, mentorship, learning progress, assessments, notifications and analytics. Additional caching, search, messaging, AI and external notification services are designed as extensible components for future integration.

The project is organized around five development milestones completed over an eight-week lifecycle: requirements and architecture, backend and persistence, frontend and dashboard development, intelligence and integration workflows, and final validation/deployment preparation. The platform is designed as a closed-loop system: assess capability, compare it with the role benchmark, calculate the gap, prioritize the gap, recommend an intervention, monitor learning, reassess capability and update the organizational knowledge picture.

Executive Area	Summary
Problem	Fragmented skill information and limited visibility into role-specific capability gaps.
Solution	Integrated skill, competency, assessment, gap-analysis and learning platform.
Primary users	Employee, Manager, HR/Admin.
Core intelligence	Proficiency comparison, severity scoring, heatmaps and recommendations.
Architecture	React.js + Spring Boot + PostgreSQL with extensible integration services.
Primary outcome	Evidence-driven learning intervention and continuous capability visibility.

Table of Contents

Table of Contents

1. Project Overview and Problem Definition----------4
2. Objectives, Scope and Stakeholders----------5
3. System Requirements----------6
4. Technology Stack and Development Environment----------7
5. Architecture and Technical Design----------8
6. Database and ER Design----------9
7. Module 1 – Authentication and RBAC----------11
8. Module 2 – Employee Profile and Skills----------12
9. Module 3 – Competency Framework----------14
10. Module 4 – Knowledge Gap Analysis----------15
11. Module 5 – Training Recommendations----------17
12. Module 6 – Mentorship and Knowledge Sharing----------19
13. Module 7 – Learning Progress----------20
14. Module 8 – Assessment and Survey----------21
15. Module 9 – Notifications----------23
16. Module 10 – Analytics Dashboards----------24
17. Module 11 – Reports and Export----------26
18. Module 12 – Integration, Testing and Deployment----------27
19. Testing and Validation----------28
20. Security and Code Quality----------29
21. Challenges and Future Enhancements----------30
22. Project Milestones, Metrics and Conclusion----------32

1. Project Overview & Problem Definition

1.1 Introduction

Modern organizations depend on specialized knowledge distributed across employees and departments. As technologies, products and business processes evolve, the required competency profile for a role also changes. Without a systematic mechanism for comparing required and actual capability, organizations may discover critical gaps only after project delays, quality problems or repeated training requests.

The Organizational Knowledge Gap Intelligence Platform addresses this challenge by establishing a common data model for employees, skills, roles, assessments and learning interventions. It transforms raw capability data into interpretable intelligence that can be consumed at three organizational levels: individual, managerial and administrative.

1.2 Problem Definition

- Employee skills may be stored in disconnected systems or maintained manually.
- Role descriptions do not always translate into measurable competency benchmarks.
- Training catalogs often recommend courses without understanding the employee's most important gaps.
- Managers require visual team-level information to prioritize interventions.
- HR requires cross-department analysis to identify strategic capability shortages.
- Learning outcomes are not always connected back to the original gap that caused the training recommendation.

1.3 Proposed Solution

The platform establishes a measurable competency pipeline. Each role has benchmark skills and required proficiency levels. Each employee has actual proficiency evidence obtained from profiles, assessments and learning outcomes. A gap-analysis service compares these values and creates severity-ranked gaps. Recommendation logic then maps those gaps to suitable training or mentorship actions. Progress and reassessment close the loop.

2. Objectives, Scope & Stakeholders

2.1 Core Objectives

- Build a responsive and role-aware React.js web interface.
- Implement secure Spring Boot REST APIs for platform operations.
- Maintain normalized employee, skill, competency, assessment and training data.
- Calculate individual and department knowledge gaps.
- Prioritize gaps using severity and organizational context.
- Generate adaptive training recommendations.
- Support peer knowledge sharing and mentorship matching.
- Track learning progress and training completion.
- Provide employee-level and organizational analytics dashboards.
- Provide reporting, export, testing and deployment readiness.

2.2 Project Scope

In Scope	Out of Scope / Future
Employee profiles and skills	Full HR payroll management
Competency benchmarks	Recruitment ATS functionality
Gap analysis and heatmaps	Financial accounting
Training and learning progress	External LMS replacement
Assessments and surveys	Biometric attendance
Notifications	Unrelated enterprise communication
Analytics and reporting	Predictive workforce planning in initial release

2.3 Stakeholders

Stakeholder	Needs
Employee	Understand personal gaps and receive useful learning actions.
Manager	Identify team capability risks and prioritize interventions.
HR/Admin	Monitor organizational competency and department trends.
System Administrator	Maintain users, roles, configuration and infrastructure.
Leadership	Review aggregated capability indicators and strategic risks.

3. System Requirements

3.1 Functional Requirements

- User registration and authentication.
- Role-based authorization for protected operations.
- Employee profile creation and editing.
- Skill catalog and proficiency management.
- Role benchmark and competency configuration.
- Individual and departmental gap analysis.
- Severity classification and heatmap generation.
- Training search and recommendation.
- Enrollment and progress tracking.
- Skill assessments and quizzes.
- Mentorship and expert discovery.
- In-app notification workflows.
- Interactive analytics dashboards.
- Analytics and reporting.
- Audit, validation and integration support.

3.2 Non-Functional Requirements

Attribute	Requirement
Security	Authenticated access, authorization, secure credentials, protected APIs.
Performance	Fast dashboard queries and low-latency common operations.
Scalability	Support growth in users, skills, assessments and events.
Availability	Failure isolation and recoverable asynchronous workflows.
Maintainability	Modular code and separation of concerns.
Usability	Responsive UI and clear visualizations.
Reliability	Validation, retries, error handling and transaction consistency.
Extensibility	Provider-independent integration boundaries.

3.3 User Stories

- As an employee, I want to see my skill gaps so that I know what to learn next.
- As a manager, I want a team heatmap so that I can identify critical capability shortages.
- As an HR administrator, I want department comparisons so that I can prioritize organization-wide interventions.
- As an administrator, I want secure role-based access so that sensitive analytics are protected.
- As a learner, I want progress tracking so that completed training updates my capability journey.

4. Technology Stack & Development Environment

4.1 Frontend

Technology	Role in Platform
JavaScript	Application programming language.
React.js	Component-based user interface.
React Router	Navigation and protected route handling.
Axios	HTTP API communication.
Tailwind CSS	Responsive utility-first styling.
Chart.js	Standard analytical charts.
Recharts	React-oriented dashboard charts.
D3.js	Custom data-driven visualizations.
Context API	Shared authentication/application state.

4.2 Backend

Technology	Role in Platform
Java	Backend programming language.
Spring Boot	Application framework.
Spring Security	Authentication and authorization.
Spring Data JPA	Repository and persistence abstraction.
Hibernate	Object-relational mapping.
Maven	Build and dependency management.
Spring WebSocket	Real-time communication.

4.3 Data, Messaging and Planned Integrations

Technology	Role
PostgreSQL	Primary relational database.
Redis	Caching and low-latency access.
Elasticsearch	Skill and expert search.
Kafka / RabbitMQ	Asynchronous messaging.
OpenAI API / LLM	Adaptive recommendation generation.
Twilio SMS	SMS notification channel.
Firebase Cloud Messaging	Push notification channel.
JavaMailSender	Email notification channel.

5. System Architecture

The system uses layered boundaries to isolate presentation, security, domain logic, persistence and external integration. The client communicates through a controlled API boundary. Backend services perform validation and business operations, while asynchronous operations are delegated to messaging consumers where appropriate.

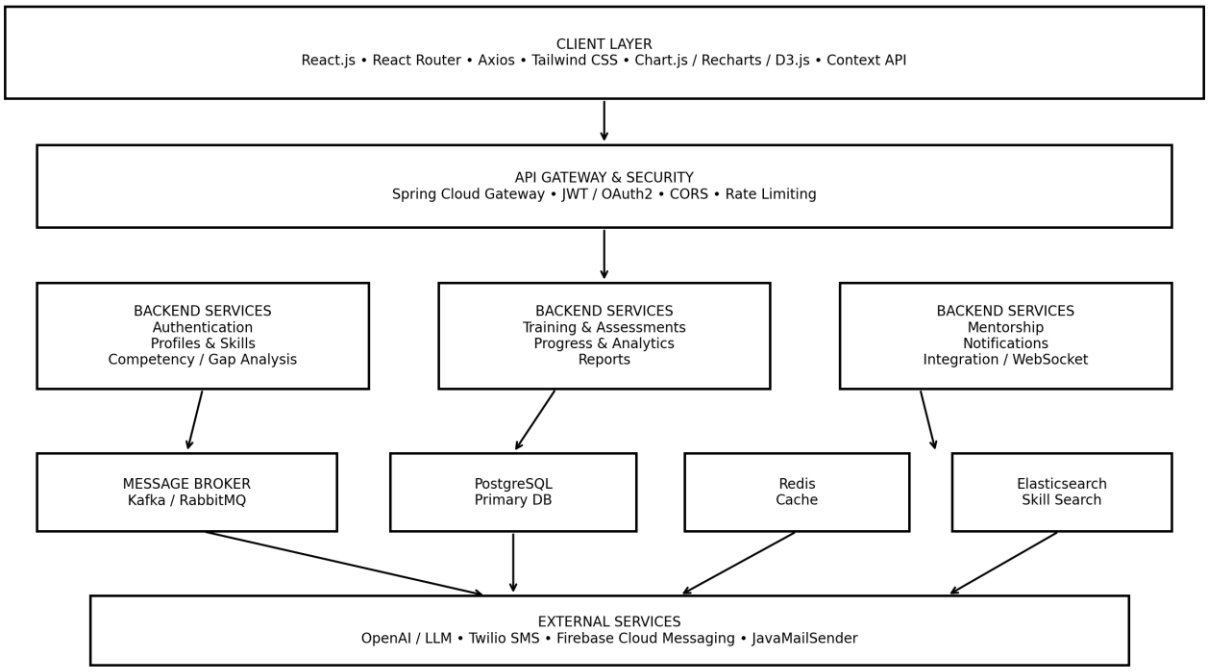


Figure 5.1 – System Architecture

5.1 Architecture Responsibilities

Layer	Responsibility
Client	Interaction, forms, dashboards, charts and route protection.
Gateway	Routing, token validation, CORS and rate limiting.
Application services	Domain-specific operations and business rules.
Persistence	Transactional storage and relational consistency.
Cache	Frequently requested or computed information.
Search	Fast skill/expert discovery.
Broker	Decoupled asynchronous event delivery.
External services	AI recommendations and notification delivery.

6. Database Design & ER Model

PostgreSQL is the primary source of truth. The database is organized around normalized entities that reduce duplication and provide clear foreign-key relationships. Skills are reusable records rather than free-text values, allowing consistent analysis across profiles, benchmarks and training.

Figure 6.1 – Relational Entity-Relationship Model

6.1 Core Tables

Table	Key Fields	Purpose
users	user_id PK, email UNIQUE, role	Identity and access.
employee_profile	profile_id PK, user_id FK	Employee organizational information.
skills	skill_id PK, skill_name	Normalized skill catalog.
employee_skill	employee_skill_id PK, profile_id FK, skill_id FK	Employee proficiency.
role_benchmark	benchmark_id PK	Role competency baseline.
benchmark_skill	id PK, benchmark_id FK, skill_id FK	Required skill and level.
knowledge_gap	gap_id PK, profile_id FK, skill_id FK	Computed capability gap.
training	training_id PK, skill_id FK	Learning intervention catalog.

enrollment	enrollment_id PK, user_id FK, training_id FK	Learning progress.
assessment	assessment_id PK, user_id FK	Capability evidence.
notification	notification_id PK, user_id FK	Alert history.

6.2 Integrity Rules

- Email values must be unique.
- Foreign keys must reference valid parent records.
- Employee-skill assignments should not be duplicated.
- Proficiency values must remain within the defined scale.
- Enrollment progress must remain between 0 and 100 percent.
- Completed enrollment should have a completion timestamp.
- Deleted or inactive skills should not invalidate historical assessment records.

7. Module 1 – Authentication & RBAC

7.1 Module Purpose

Authentication establishes user identity; authorization establishes what that identity is allowed to access. The module is implemented around Spring Security and is designed for JWT-based stateless API access, with OAuth2 as an enterprise integration option.

7.2 Main Workflow

- Client submits registration or login request.
- Backend validates input and account state.
- Credentials are authenticated using secure password handling.
- An access token is issued for a successful authentication.
- Client includes the token with protected API calls.
- Security filters validate the token before controller execution.
- Role-based rules determine whether the requested operation is permitted.

7.3 API Examples

POST /api/auth/register
Content-Type: application/json

```
{
 "email": "employee@example.com",
 "password": "********"
}
```

GET /api/auth/user
Authorization: Bearer <token>

7.4 Security Rules

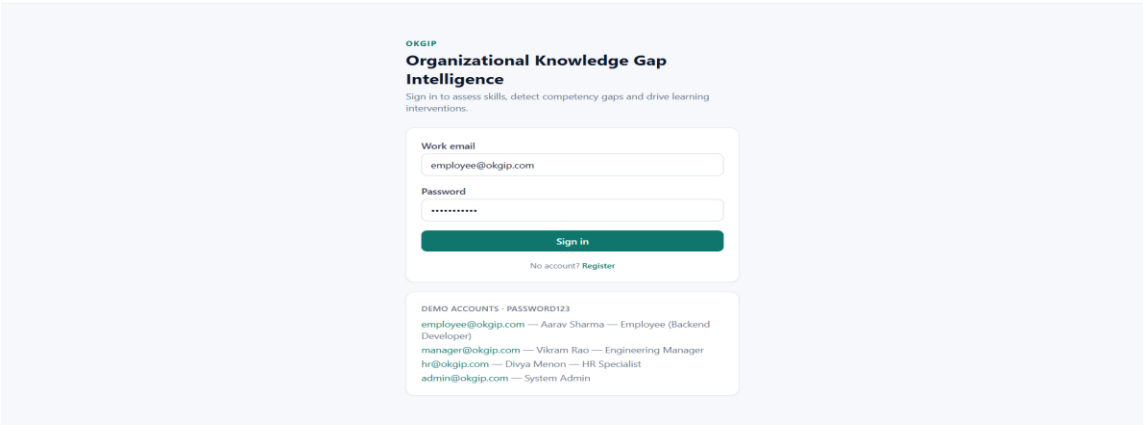
- Do not expose password hashes in API responses.
- Use HTTPS in production.
- Expire tokens according to the security policy.
- Restrict administrative endpoints to authorized roles.
- Use secure configuration for secrets and OAuth2 credentials.
- Apply CORS and rate limits at the API boundary.

8. Module 2 – Employee Profile & Skill Inventory

8.1 Purpose

The employee profile is the central capability record. It combines identity-linked organizational information with a structured list of skills and proficiency values. The design prevents skill names from becoming inconsistent free-text entries.

- Working login and registration screenshots.



8.2 Proficiency Model

Level	Score	Meaning
Unaware	0	No meaningful awareness of the skill.
Beginner	1	Basic familiarity; requires guidance.
Intermediate	2	Can perform common tasks independently.
Advanced	3	Strong practical capability and problem solving.
Expert	4	Deep expertise; can mentor or lead others.

8.3 Main Operations

- Create and update employee profile.
- Retrieve employee details by authorized identifier.
- Add a skill to the employee inventory.
- Update proficiency after assessment or learning.
- Remove or deactivate obsolete skill assignments.
- Display skill history and current proficiency.

8.4 Data Validation

```
if proficiency < 0 || proficiency > 4:
    reject("Invalid proficiency level")

if employeeSkillExists(employeeId, skillId):
```

```
    updateExistingAssignment()
else:
    createAssignment()
```

9. Module 3 – Competency Framework & Role Benchmarking

9.1 Purpose

The competency framework converts job expectations into measurable skill requirements. A benchmark identifies the skills required for a role and assigns a target proficiency. This benchmark becomes the reference point for gap analysis.

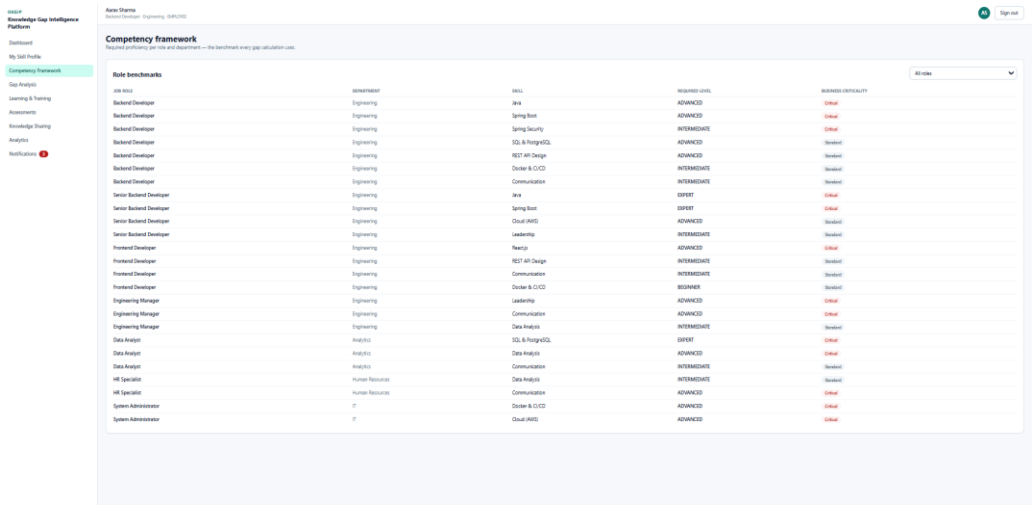


Figure 10.1 — Competency Framework and Role Benchmarking

9.2 Benchmark Workflow

- Administrator creates or selects a role.
- Required skills are attached to the role benchmark.
- Each skill receives a required proficiency level.

- Employees assigned to the role inherit the benchmark context.
- Gap analysis compares current proficiency against the benchmark.
- Benchmark versions can be retained when role requirements change.

9.3 Example

Skill	Required	Employee	Gap
Java	4	3	1
SQL	3	2	1
React	3	3	0
System Design	4	1	3
Communication	3	2	1

9.4 Benchmark Governance

- Only authorized administrators should modify official role benchmarks.
- Benchmark changes should be timestamped and auditable.
- Historical gap calculations should preserve the benchmark context used at calculation time.
- Inactive roles should remain available for historical reporting but should not be assigned to new employees.

10. Module 4 – Knowledge Gap Analysis

10.1 Purpose

The gap-analysis module is the intelligence core of the platform. It transforms proficiency evidence into comparable severity information for employees, teams and departments.

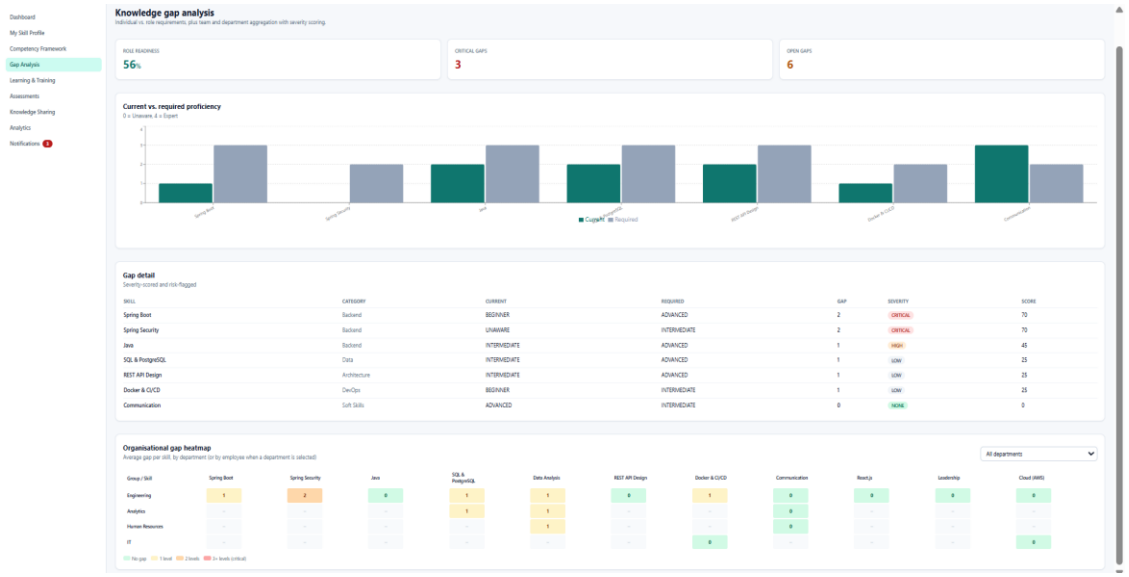


Figure 10.1 — Individual Knowledge Gap Analysis

10.2 Calculation

```
gap = max(0, requiredProficiency - actualProficiency)
normalizedGap = gap / max(requiredProficiency, 1)

if normalizedGap >= 0.75:
    severity = "CRITICAL"
elif normalizedGap >= 0.50:
    severity = "HIGH"
elif normalizedGap >= 0.25:
    severity = "MEDIUM"
else:
    severity = "LOW"
```

10.3 Heatmap Generation

The heatmap dataset aggregates employee or department rows against skill columns. Each cell contains either the gap score or severity category. Color intensity is then applied by the frontend visualization layer.

Dimension	Aggregation
Employee	Average and maximum skill gap.
Department	Average gap by skill and critical-gap count.

Role	Gap distribution against role benchmark.
Skill	Number of affected employees and severity distribution.
Time	Change in gap after assessments and learning interventions.

10.4 Real-Time Refresh

When an assessment or proficiency update changes a skill, the system should invalidate affected cached results and trigger recalculation. WebSocket or event-driven updates can refresh dashboards without requiring the user to manually reload the page.

11. Module 5 – Training Recommendation

11.1 Purpose

The recommendation module converts knowledge gaps into practical learning interventions. A deterministic rule-based layer provides predictable baseline recommendations, while an LLM integration can refine the result using contextual information.

11.2 Recommendation Inputs

- Employee role and department.
- Skill with the highest priority gap.
- Required and actual proficiency.
- Gap severity.
- Previous training and completion history.
- Preferred learning format where available.
- Training difficulty and skill mapping.
- Current workload or availability where supported.

11.3 Adaptive Recommendation Logic

```
context = {
  role, department,
  skillGap, severity,
  currentLevel,
  completedTraining,
  learningHistory
}

candidates = findTraining(skillGap)
candidates = removeCompleted(candidates, completedTraining)
ranked = rankByRelevance(candidates, context)
recommendation = rankAndSelect(ranked, context)
```

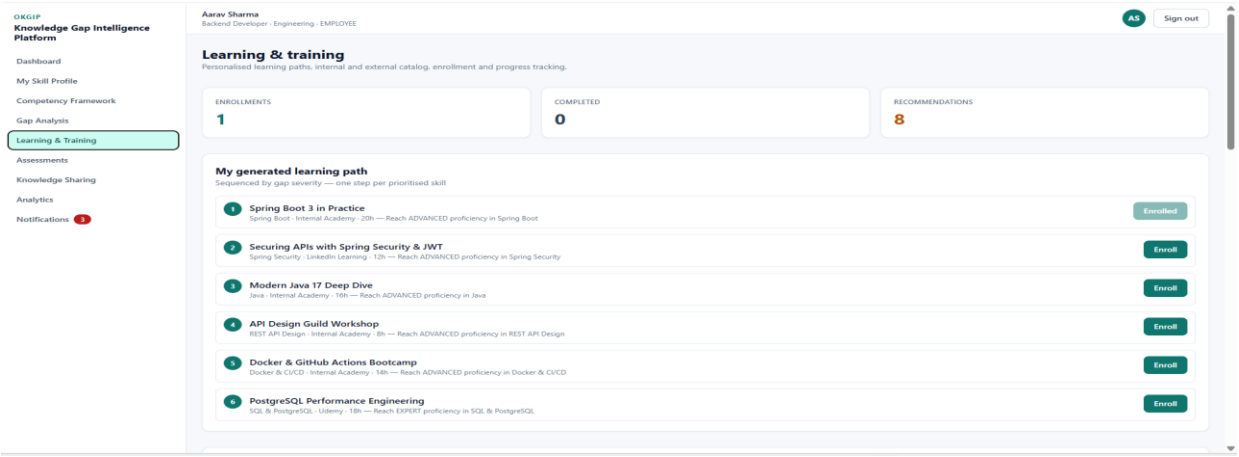


Figure 11.1 — Training Recommendations

11.4 Quality Metrics

Metric	Meaning
Recommendation acceptance	How often users accept suggestions.
Completion rate	How often accepted training is completed.
Gap reduction	Change in targeted gap after intervention.
Repetition rate	Frequency of repeated irrelevant recommendations.
Time to recommendation	Latency from gap detection to recommendation availability.

12. Module 6 – Knowledge Sharing & Mentorship

12.1 Purpose

Not every knowledge gap requires formal training. The mentorship module uses complementary skills to connect employees who can learn from one another. This creates an internal knowledge network and reduces dependency on external courses for every capability need.

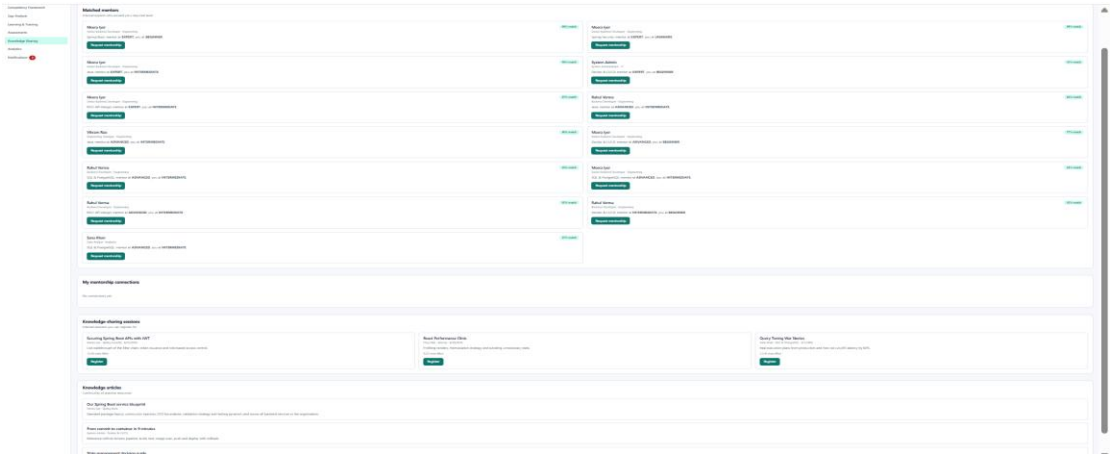


Figure 13.1 — Mentor Matching and Knowledge Sharing

12.2 Matching Model

skillMatch = overlap(mentor.skills, requiredSkills)
roleFit = compareRoles(mentor.role, mentee.role)
complementarity = mentorLevel - menteeLevel
matchScore = weighted(skillMatch, roleFit, complementarity, availability)

12.3 Workflow

- Identify an employee with a meaningful knowledge gap.
- Extract target skills and required proficiency.
- Search employees with high proficiency in those skills.
- Filter by role compatibility, department rules and availability.
- Rank candidates using a matching score.
- Create mentorship request or suggestion.

- Track accepted, active and completed mentorship relationships.

12.4 Governance

- Employees should control whether they are available as mentors.
- Sensitive employee information should not be exposed during matching.
- Matching should not create discriminatory or unfair selection criteria.
- Administrators should be able to deactivate inappropriate matches.
- Mentorship outcomes can contribute to organizational knowledge-sharing analytics.

13. Module 7 – Learning Progress Tracking

13.1 Purpose

Learning progress closes the loop between recommendation and measurable intervention. Every enrollment records status, progress and relevant dates.

13.2 Status Model

Status	Meaning
NOT_STARTED	Enrollment exists but learning has not begun.
IN_PROGRESS	Learner is actively progressing.
COMPLETED	Required learning is completed.
PAUSED	Learner temporarily stopped progress.
CANCELLED	Enrollment was cancelled.

13.3 Learning Progress Analytics

learningDays = max(1, completionDate - startDate)
velocity = completedUnits / learningDays

Velocity should be interpreted carefully. A higher velocity does not automatically mean better learning; completion quality and post-training assessment results should also be considered.

13.4 Feedback to Gap Analysis

- Training completion can trigger a reassessment recommendation.
- Post-training assessment can update proficiency.
- Updated proficiency changes the knowledge-gap score.
- Reduced gaps can lower recommendation priority.
- Unchanged gaps can trigger alternative learning or mentorship suggestions.

14. Module 8 – Assessment & Survey

14.1 Purpose

Assessment provides evidence of actual capability. The module supports multiple perspectives because self-assessment alone may be subjective. Manager and peer assessments provide additional evidence for a more reliable capability estimate.

14.2 Skill Assessment Types

Type	Evidence Source	Use
Self	Employee	Reflection and initial skill estimate.
Manager	Manager	Role-performance perspective.
Peer / 360	Colleagues	Collaborative capability evidence.

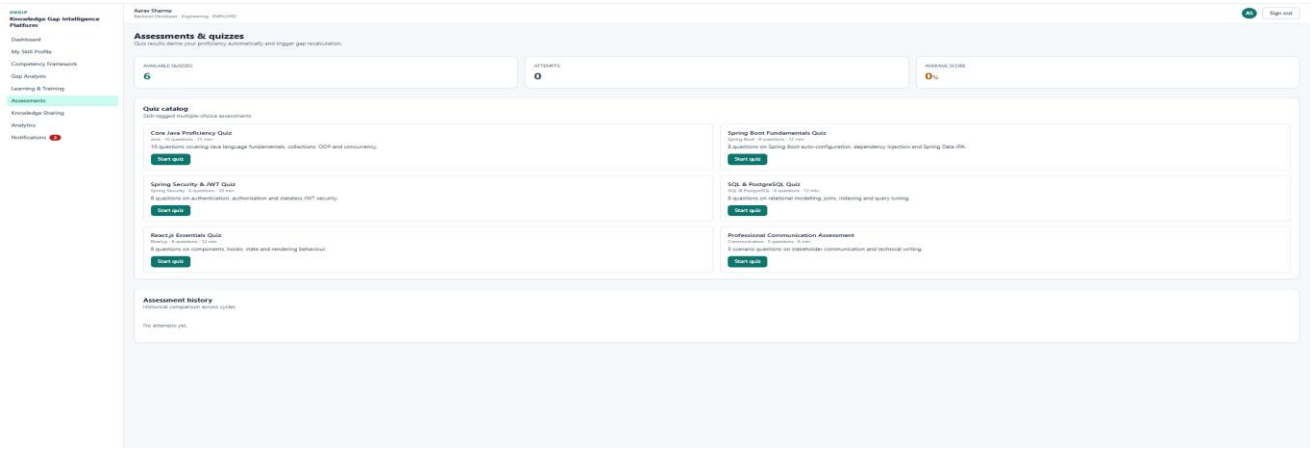
Post-training	Assessment after learning	Measure intervention effect.

14.3 Weighted Scoring

weightedScore = sum(score_i * weight_i) / sum(weight_i)

14.4 Assessment Controls

- Prevent unauthorized users from submitting assessments for unrelated employees.
- Validate score ranges.
- Prevent accidental duplicate submissions where the workflow requires one response.
- Record timestamps and evaluator relationships.
- Maintain historical results for trend analysis.
- Use anonymization or aggregation for peer feedback where required.



Skill Assessment and Quiz Interface

15. Module 9 – Notification System

15.1 Purpose

Notifications make intelligence actionable. The notification service receives business events and translates them into user-facing messages. Asynchronous processing is preferred so that notification provider latency does not delay core transactions.

15.2 Notification Channels

Channel	Example Use
In-app / WebSocket	Real-time critical-gap or dashboard updates.
Email	Training reminders, weekly summaries and reports.
SMS	High-priority operational alerts.
Push	Immediate mobile/browser learning notifications.

15.3 Event Flow

AssessmentCompleted
↓
GapRecalculated
↓
CriticalGapDetected
↓
MessageBroker
↓
NotificationConsumer
↓
ChannelAdapter
↓
DeliveryStatus

15.4 Reliability

- Use retry policies for temporary provider failures.
- Use dead-letter handling for persistent failures.
- Record delivery status.
- Use idempotency to avoid duplicate notifications.
- Do not expose provider credentials to the frontend.
- Allow users to configure notification preferences where applicable.

16. Module 10 – Analytics Dashboards

16.1 Dashboard Views

The demonstrated interface provides employee-level capability and learning information together with organizational analytics such as readiness, gaps, heatmaps and training insights.

Role	Dashboard Focus
Employee	Personal skills, gaps, recommendations, learning progress and assessments.
Manager	Team heatmap, critical skills, role coverage, training progress and interventions.
HR/Admin	Department comparison, organization-wide competency trends, critical gaps and reports.

16.2 Key Metrics

Metric	Formula / Interpretation
Average gap	Mean normalized gap across selected population.
Critical gap count	Number of skills classified as critical.
Skill coverage	Employees meeting or exceeding required level / applicable employees.
Completion rate	Completed learning / started learning.
Assessment trend	Change in assessed proficiency over time.

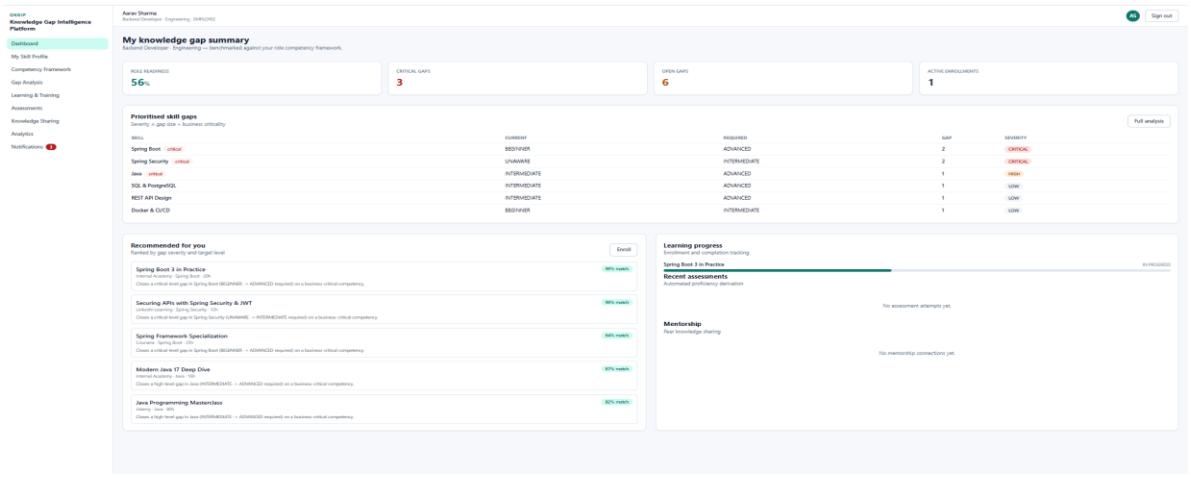
Learning velocity	Learning units completed per active learning day.

16.3 Visualization Design

- Heatmaps show concentration of skill gaps.
- Bar charts compare departments or roles.
- Line charts show gap and proficiency trends.
- Progress charts show learning completion.
- Filters allow users to select department, role, skill and time range.
- Tooltips should expose the underlying metric without overwhelming the user.

16.4 Dashboard Data Flow

React Dashboard
→ Axios API request
→ Gateway / Security
→ Analytics Service
→ PostgreSQL / Redis
→ Aggregated JSON
→ Chart / Heatmap component



Employee Analytics Dashboard

17. Module 11 – Analytics & Reports

17.1 Purpose

Reports convert interactive analytics into durable management artifacts.

17.2 Report Types

Report	Contents
Employee Capability Report	Profile, skills, gaps, recommendations and learning progress.
Department Gap Report	Skill severity distribution, heatmap summary and critical gaps.
Training Report	Enrollment, completion, status and velocity.
Assessment Report	Assessment scores and trend summaries.
Management Summary	High-level capability indicators and intervention priorities.

17.3 Reporting Workflow

- User selects report type and filters.
- Backend validates access rights.
- Analytics data is queried or aggregated.
- Report template is populated.
- PDF/Excel document is proposed/extensible.
- Small reports can be returned immediately; large reports can be generated asynchronously.
- Audit information records who generated the report and when.

17.4 Example Request

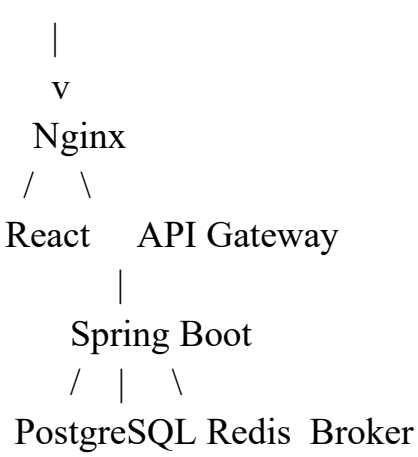
```
POST /api/reports/export
{
 "type": "DEPARTMENT_GAP",
 "departmentId": 12,
 "format": "PDF",
```

```
  "from": "2026-08-01",
  "to": "2026-08-31"
}
```

18. Module 12 – Integration, Testing & Deployment Readiness

18.1 Deployment Architecture

The proposed deployment design uses containerization to isolate application and infrastructure responsibilities. Nginx can serve static frontend content and reverse-proxy API traffic. Docker Compose is useful for reproducible development and staging environments.



18.2 Container Responsibilities

Container	Responsibility
frontend	Build and serve React application.
backend	Run Spring Boot APIs and business logic.
postgres	Persistent relational database.
redis	Cache and temporary data.
broker	Asynchronous events.
nginx	Reverse proxy and static delivery.

18.3 Deployment Checklist

- Build frontend production bundle.
- Build backend artifact.
- Create production containers.
- Configure environment variables.
- Configure database connection and migrations.
- Configure CORS and allowed origins.
- Configure HTTPS at the reverse proxy.
- Run health checks.
- Run automated tests.
- Verify authentication and critical API flows.
- Back up database and verify restore procedure.

19. Testing, Validation & Code Quality

19.1 Test Strategy

Level	Technology	Focus
Unit	JUnit + Mockito	Business rules and isolated services.
Integration	Spring Boot Test	Controller/service/repository interaction.
Frontend	React Testing Library	Components, forms and user interaction.
API	Postman	Security, status codes, CORS and rate limiting.
Build	Maven / npm	Compilation and dependency validation.

19.2 Representative Test Cases

ID	Test	Expected
TC01	Valid registration	Account created.
TC02	Duplicate email	Rejected.
TC03	No JWT	401 Unauthorized.
TC04	Wrong role	403 Forbidden.

TC05	Assessment submission	Stored and processed.
TC06	Gap calculation	Correct positive gap.
TC07	Training completion	Progress reaches 100%.
TC08	Critical gap	Recommendation event generated.
TC09	Rate limit exceeded	Request throttled.
TC10	Invalid CORS origin	Request blocked.
TC11	Dashboard filtering	Correct filtered metrics.
TC12	Report generation	Correct output format.

19.3 Code Quality

- Use controller-service-repository separation.
- Keep business rules testable and isolated.
- Centralize exception handling.
- Validate request payloads.
- Use DTOs where API/persistence separation is required.
- Keep secrets outside source control.
- Use meaningful naming and consistent formatting.
- Review database queries for unnecessary loading and N+1 behavior.

20. Security Design, API Validation & Production Readiness

20.1 Security Controls

- JWT authentication for stateless API access.
- Role-based authorization for sensitive operations.
- OAuth2-ready enterprise identity integration.
- CORS allow-list at gateway.
- Rate limiting for abuse-prone endpoints.
- HTTPS for production traffic.
- Secure storage of secrets and provider credentials.
- Input validation and output sanitization.

- Audit logging for sensitive administrative operations.

20.2 Postman Validation Plan

Validation	Procedure	Expected Evidence
Authentication	Login and use returned token.	Successful protected request.
Authorization	Use employee token on admin endpoint.	403 response.
CORS	Send request from disallowed origin.	CORS rejection.
Rate limit	Send repeated requests rapidly.	429/throttled response.
Validation	Send malformed payload.	4xx validation response.
Error handling	Request unknown resource.	Controlled error response.

20.3 Production Readiness Checklist

- Environment-specific configuration.
- Database backup strategy.
- Health endpoints.
- Centralized logging.
- Monitoring and alerts.
- Error tracking.
- Dependency vulnerability review.
- Secure reverse proxy configuration.
- Rollback procedure.
- Data retention and privacy policy.

21. Challenges, Future Enhancements & Scalability

21.1 Technical Considerations and Mitigation Strategies

Challenge	Engineering Response

Kafka/RabbitMQ reliability	Retries, idempotency and dead-letter handling.
Elasticsearch query cost	Indexes, filters, pagination and optimized mappings.
LLM latency	Async requests, caching and non-blocking workflows.
OAuth2 redirect configuration	Exact environment-specific HTTPS redirect URIs.
Redis stale data	Targeted cache invalidation after source updates.
Notification failures	Retry queues and delivery status tracking.
Analytics load	Aggregation, caching and scheduled computation.

21.2 Future Enhancements

- Semantic skill ontology and synonym normalization.
- Graph-based expert and mentor discovery.
- Predictive skill-demand forecasting.
- Adaptive learning-path optimization.
- Multilingual recommendation and notification support.
- Advanced observability with logs, metrics and distributed tracing.
- Enterprise identity provisioning.
- Large-scale service decomposition based on measured load.
- Recommendation quality evaluation and A/B testing.
- Automated competency benchmark suggestions based on organizational trends.

21.3 Scalability Strategy

Component	Scale Strategy
Spring Boot	Horizontal scaling behind load balancer.
PostgreSQL	Indexing, read replicas and partitioning when justified.
Redis	Clustered cache and controlled TTL.
Elasticsearch	Shards, replicas and optimized mappings.
Kafka	Topic partitioning and consumer scaling.
Frontend	CDN, caching and code splitting.
Reports	Background jobs for large exports.

22. Project Milestones, Metrics, Conclusion & References

22.1 Eight-Week Milestones

Milestone	Weeks	Deliverables
1. Planning & Architecture	1–2	Requirements, use cases, architecture and data model.
2. Backend Foundation	2–3	Spring Boot, entities, repositories and security foundation.
3. Frontend & Dashboards	4–5	React UI, routing, profiles, skills and analytics.
4. Intelligence & Integrations	6–7	Gap analysis, recommendations, assessments and integrations.
5. Testing & Deployment	8	Validation, deployment preparation and documentation.

22.2 Evaluation Metrics

Metric	Definition	Final Value
API success rate	Successful API calls / total calls	Project test results
Test pass rate	Passed tests / total tests	Test execution
Critical gap reduction	Before-vs-after critical gaps	Gap analysis
Recommendation acceptance	Accepted / presented recommendations	Application data
Training completion	Completed / started training	Learning module
Average gap calculation time	Mean calculation latency	Performance test

22.3 Conclusion

The Organizational Knowledge Gap Intelligence Platform provides a structured, measurable and extensible solution for enterprise capability intelligence. By linking employees, skills, role benchmarks, assessments, gap severity, recommendations and learning progress, the platform creates a continuous feedback loop rather than a static training catalog.

The architecture is intentionally modular and is designed to support secure APIs, relational persistence, analytics, real-time communication, asynchronous messaging, caching, search and external AI/notification services. The design can therefore evolve from a project prototype into a scalable enterprise platform as actual usage data, performance measurements and operational requirements become available.

22.4 References

- Spring Boot Reference Documentation.
- Spring Security Reference Documentation.
- Spring Data JPA Reference Documentation.
- PostgreSQL Documentation.
- React Documentation.
- React Router Documentation.
- Axios Documentation.
- Tailwind CSS Documentation.
- Apache Kafka Documentation.
- RabbitMQ Documentation.
- Redis Documentation.
- Elasticsearch Documentation.
- OpenAI API Documentation.
- JUnit 5 User Guide.
- Mockito Documentation.
- React Testing Library Documentation.
- Docker Documentation.